

YouTube AI Shorts Platform

Leadership Architecture Brief

Audience: Senior Management, Enterprise Architect, Development Manager

Date: 2026-06-28

1) Executive Snapshot

What this platform does

A fully automated content factory that produces and publishes vertical YouTube Shorts twice daily.

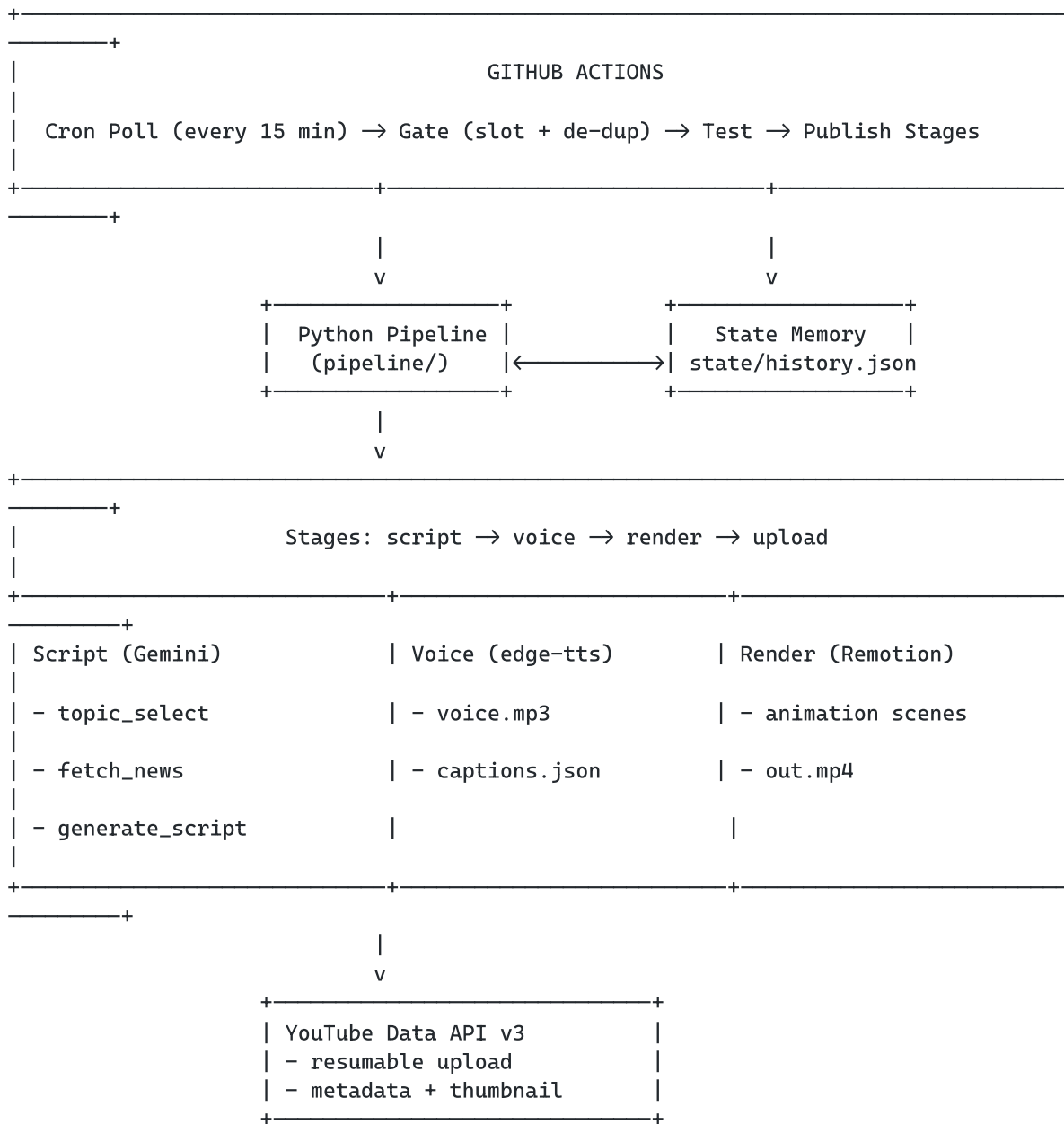
Why this matters

- Predictable publishing cadence with minimal human dependency
- Lower operational cost through free/open tooling
- Faster time-to-market from idea to published short
- Repeatable and auditable process via CI workflow controls

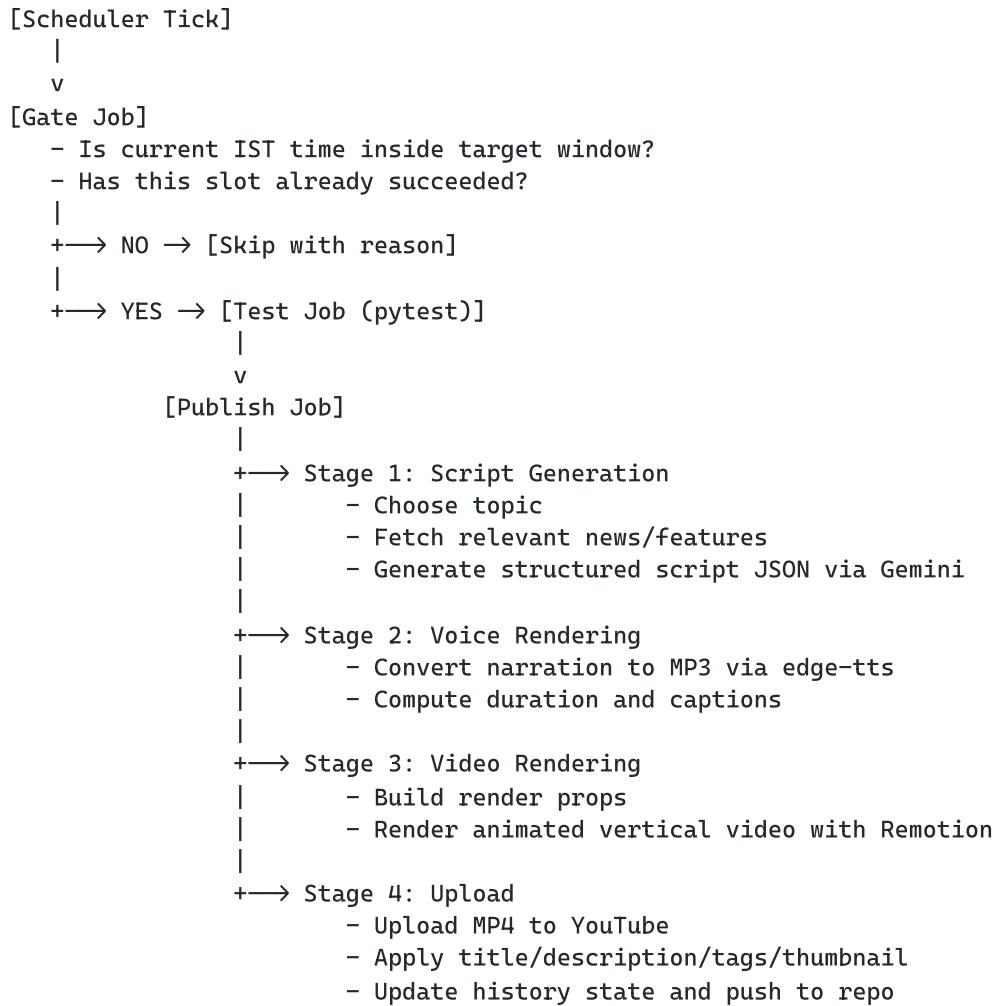
Business outcome

- Two publish slots per day: 09:00 IST and 21:00 IST
 - Automated de-duplication to prevent accidental double publishing
 - End-to-end flow from topic selection to YouTube upload
-

2) Architecture at a Glance



3) End-to-End Operational Flow



4) Topic Selection Design

Topic catalog

- ai_news
- copilot
- claude
- cursor

Selection modes

1. rotate (default)

- Deterministic daily selection based on date ordinal
- Guarantees balanced coverage across topic set

2. trending (optional)

- Computes per-topic score from fresh item counts
- Applies cooldown penalty to recently used topics
- Can include analytics weights from historical performance

Why this is architecturally sound

- Deterministic baseline avoids topic starvation
 - Trending mode enables adaptive behavior when needed
 - Cooldown prevents repetitive user experience
-

5) Feature/News Fetching and Relevance Pipeline

```
Topic Definition
  → RSS feed list
  → Google News query list
  → must_include keywords (for narrow topics)

Fetch Layer
  → Request with timeout + User-Agent
  → Parse feed entries
  → Capture title, summary, link, published date, optional image

Quality Layer
  → Freshness filter
  → De-dup by normalized title
  → must_include filter (fallback to full pool if empty)

Output
  → ranked candidate list for script generation
  → optional hero image candidate for visuals
```

Resilience controls:

- Feed-level failures are isolated (single source outage does not fail pipeline)
 - Typed error raised only if entire candidate pool is unusable
-

6) Script Generation: Framework and AI

AI stack used

- SDK: google-genai
- Model: gemini-2.5-flash (with fallback chain)

Prompt construction inputs

- Selected topic focus
- Fresh source items (title/summary/link/date)
- Anti-repetition memory from recent history
- Hook style guidance
- Audience clarity constraints

Structured output contract

The model must return strict JSON containing:

- title
- title_variants
- description
- tags
- lines (spoken script)
- points (visual cards)
- flow (animated process steps)

Why this design works

- Structured output minimizes post-processing ambiguity
 - Built-in validation protects downstream stages
 - Output format is directly consumable by rendering system
-

7) Voice Rendering and Audio Intelligence

Technology

- edge-tts for synthesis
- mutagen for reliable MP3 duration

Outputs

- build/voice.mp3
- build/captions.json (word boundaries + duration)

Controls

- Retry for transient synthesis issues
- Empty audio rejection
- Minimum duration guard to prevent low-quality outputs

Current default voice profile

- en-US-JennyNeural
- Tuned for energetic, clear pronunciation

8) Video Composition, Animation, and Rendering

Rendering framework

- Remotion 4 (React + TypeScript)

Composition profile

- 1080 x 1920 (vertical)
- 30 FPS
- Duration derived from audio duration

Animation/visual components

- Dynamic gradient background with topic-driven accents
- Topic chip and hook transitions
- Highlight card sequences
- Flow diagram scene
- Optional hero image overlay

Data contract between Python and Remotion

- build/render-props.json is the typed handoff artifact
- zod schema validates props before render

Performance profile (free-runner optimized)

- Tuned codec settings (h264 + veryfast preset)
- Safe scaling policy for codec-compatible dimensions
- Concurrency controls for CI runtime balance

9) YouTube Upload and Publishing

Upload mechanism

- YouTube Data API v3 (OAuth refresh token)
- Resumable upload with retry/backoff

Metadata operations

- Title, description, tags from generated script
- Privacy mode control from environment
- Best-effort thumbnail assignment

Completion state

- video_id and publish timestamp persisted to history.json

- history committed back to git for auditability and anti-repetition memory

10) Twice-Daily Automation with GitHub Actions

Scheduler: every 15 minutes

Slot windows (IST):

- Morning slot: 09:00 → 12:00
- Evening slot: 21:00 → 00:00

Gate logic:

- 1) If outside window → skip (outside_target_window)
- 2) If already succeeded in current slot → skip (slot_already_completed)
- 3) Else → run test + publish pipeline

Why this pattern instead of exact-time cron:

- Handles scheduler drift gracefully
- Preserves exactly-once behavior per slot
- Reduces operational misses for business-critical cadence

11) Capability Map (Enterprise View)

Capability	Implementation	Outcome
Scheduling and orchestration	GitHub Actions workflow + gate	Deterministic twice-daily execution
Content intelligence	topic_select + fetch_news + generate_script	Fresh and relevant scripts
AI generation	Gemini via google-genai	Structured script assets
Voice synthesis	edge-tts + timing extraction	Natural narration + subtitle timing
Visual generation	Remotion templates and animations	Branded, modern short-form video
Distribution	YouTube API v3 resumable upload	Reliable publishing with metadata

Governance and memory	history.json committed to git	Auditability and anti-repetition
-----------------------	-------------------------------	----------------------------------

12) Repository Structure (Implementation Detail)

```
.github/workflows/
  daily-short.yml          # poll + gate + test + publish stages
  healthcheck.yml          # weekly credential checks

pipeline/
  run.py                  # stage orchestrator
  topics.py               # topic definitions, feeds, keywords, visual theme
hints
  topic_select.py         # rotate/trending strategy
  fetch_news.py           # ingestion and relevance filtering
  generate_script.py       # Gemini prompt + strict JSON handling
  tts.py                  # voice synthesis + caption timing
  render_props.py         # typed render payload builder
  upload_youtube.py       # resumable upload to YouTube
  history.py              # durable state memory
  analytics.py            # optional feedback loop from public stats

remotion/
  src/Root.tsx            # composition config
  src/Short.tsx           # animation scenes
```

13) Risk and Mitigation Summary

Risk	Potential Impact	Mitigation in Current Design
External source downtime	Weak content pool	Multi-source ingestion + Google News fallback
LLM throttling or model issues	Script generation failure	Model fallback chain + retries
TTS failures	Missing narration	Retry + output validation
Render instability	Delayed publish	Tuned render profile + codec-safe scaling
Duplicate publishing	Channel quality risk	Slot de-dup gate logic
Secret/config issues	Pipeline interruption	Preflight checks + healthcheck workflow

14) Leadership Takeaways

For Senior Management:

- This is a productionized, low-touch content engine with predictable throughput and low operating cost.

For Enterprise Architecture:

- The system uses clean stage boundaries, typed artifacts, resilient integration patterns, and auditable state transitions.

For Development Management:

- Stage isolation in CI improves observability, debugging speed, and team ownership boundaries.

15) Visual Flow Summary (Presentation Slide Style)

```
[Topic Strategy] → [News/Feature Pool] → [AI Script JSON] → [Voice + Captions]  
→ [Animated Render] → [YouTube Upload] → [History Feedback Loop]
```

This architecture is implementable, scalable for additional channels/topics, and operationally robust for daily unattended publishing.